

PROJECT REPORT

AI SAAS CHATBOT USING RETRIEVAL-AUGMENTED GENERATION

Submitted in partial fulfillment of the requirements
for the award of

Bachelor of Engineering in Computer Engineering

by

Your Name Here

Roll No: XXXX

Under the guidance of

Prof. Guide Name

Department of Computer Engineering

College Name

University Name

2025–2026

Certificate

This is to certify that the project report entitled “**AI SaaS Chatbot using Retrieval-Augmented Generation**” is a bonafide work carried out by **Your Name Here** under my supervision and guidance during the academic year 2025–2026, in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Engineering.

Guide Signature: _____

Head of Department: _____

Principal: _____

Date: _____

Place: _____

Declaration

I hereby declare that the project report entitled “**AI SaaS Chatbot using Retrieval-Augmented Generation**” is my own work carried out under the supervision of my guide. This report has not been submitted to any other university or institute for the award of any degree or diploma.

Student Signature: _____

Name: _____

Date: _____

Acknowledgement

I express my sincere gratitude to my project guide and the faculty members of the Department of Computer Engineering for their guidance, timely support, and constructive feedback throughout this work. I also thank my classmates and family for their encouragement. Their input helped in converting an initial prototype into a structured, evaluated, and production-minded Retrieval-Augmented Generation (RAG) system.

Abstract

This project presents the design and implementation of an AI SaaS chatbot backend using Retrieval-Augmented Generation (RAG). The objective is to provide accurate, context-grounded responses by combining semantic retrieval from enterprise documents with large language models (LLMs). The system supports multiple providers (Groq and Google Gemini), local embedding generation using Hugging Face embeddings, persistent vector storage through ChromaDB, and conversation memory through SQLite.

The architecture is designed for practical deployment and maintainability. It includes a structured ingestion pipeline for PDF, TXT, CSV, Excel, and JSON files; a retrieval layer with configurable chunking and top-k search; a generation layer with strict context-grounding prompts; and API endpoints implemented in FastAPI. To improve reliability, the system contains model fallback logic for quota limits and transient provider failures. To improve reproducibility and DevOps readiness, the project includes Docker containerization, automated data ingestion orchestration, observability hooks via LangSmith, and a quantitative RAG quality evaluation framework via Ragas.

A testing suite using Pytest validates ingestion behavior and core RAG flow. The result is a professional backend foundation that demonstrates the complete AI application lifecycle: data ingestion, indexing, retrieval, generation, observability, evaluation, and operational automation. The work highlights practical tradeoffs, current limitations, and a roadmap for enterprise hardening such as authentication, async ingestion queues, and hybrid retrieval with reranking.

Keywords: Retrieval-Augmented Generation, Machine Learning, Natural Language Processing, ChromaDB, LangChain, FastAPI, Ragas, LangSmith, MLOps.

Contents

Certificate	1
Declaration	2
Acknowledgement	3
Abstract	i
List of Abbreviations	vii
1 Introduction	1
1.1 Overview	1
1.2 Machine Learning Context	1
1.3 Machine Learning Strategies Relevant to This Work	2
1.3.1 Supervised Learning	2
1.3.2 Unsupervised and Self-Supervised Learning	2
1.3.3 Reinforcement and Preference Optimization (Background)	2
1.4 Problem Statement	2
1.5 Objectives	3
1.6 Scope and Deliverables	3
1.7 Report Organization	4
2 Literature Survey	5
2.1 Related Work in Retrieval-Augmented Generation	5
2.2 Document Ingestion and Chunking Strategies	5
2.3 Vector Databases for Semantic Retrieval	6
2.4 LLM Provider Strategy and Reliability	6
2.5 Observability and Evaluation Practices	6
2.6 Testing and MLOps Lifecycle	6

2.7	Research Gaps and Relevance to This Project	7
3	Methodology and System Design	8
3.1	Existing System	8
3.2	Proposed System	8
3.3	Functional Requirements	9
3.4	Non-Functional Requirements	9
3.5	Software and Hardware Requirements	9
3.5.1	Software Requirements	9
3.5.2	Hardware Requirements	10
3.5.3	Core Libraries Used	10
3.6	Programming Language Rationale	11
3.7	System Architecture	11
3.7.1	Logical Architecture View	11
3.7.2	Request-Response Runtime Sequence	11
3.7.3	Deployment Architecture with Docker	12
3.7.4	Repository and Filesystem Architecture	13
3.7.5	Data Representation Model	14
3.7.6	Industrial Complexity in the Implemented System	14
3.7.7	MLOps Lifecycle Diagram	16
3.8	Algorithmic Flow	16
3.8.1	Data Ingestion Algorithm	16
3.8.2	Retrieval and Generation Algorithm	17
3.8.3	Memory Management Algorithm	18
3.9	Module-Wise Design	18
3.9.1	Configuration Module	18
3.9.2	Ingestion Module	18
3.9.3	Vector Store Module	18
3.9.4	RAG Engine Module	18
3.9.5	API Module	18
3.9.6	Memory Module	19
3.9.7	Evaluation Module	19
3.9.8	Observability Module	19
3.9.9	Automation Modules	19
3.10	Design Decisions and Tradeoffs	19

4	Implementation, Results, and Discussion	20
4.1	Implementation Workflow and MLOps Staging	20
4.2	Container Build and Runtime Engineering	20
4.2.1	Docker Image Design	20
4.2.2	Service Orchestration Design	21
4.2.3	Operational Commands Used in Deployment	22
4.3	Data Representation and Evidence Tables	22
4.3.1	Corpus and Chunking Representation	22
4.3.2	Retrieval and Generation Performance Sheet	22
4.4	Evaluation and Observability Lens	22
4.4.1	RAG Quality Evaluation	22
4.4.2	Traceability and Incident Debugging	23
4.5	Validation Strategy	24
4.5.1	Automated Testing	24
4.5.2	Manual Validation Checklist	24
4.6	Complex Engineering Outcomes	25
4.7	Evidence Placeholders for Final Submission	25
4.8	Current Limitations and Hardening Plan	26
5	Conclusion and Future Work	28
5.1	Conclusion	28
5.2	Future Work	28
5.3	Final Statement	29
A	Deployment Runbook and Operational Evidence	31
A.1	Environment and Startup Checklist	31
A.2	Deployment Command Log Template	31
A.3	Runtime Observation Sheet	32
B	Screenshots and Submission Attachments	33
B.1	Plagiarism and Similarity Report Placeholder	33
B.2	Journal/Conference Paper Placeholder	33

List of Figures

3.1	Layered architecture used by the AI SaaS chatbot platform.	12
3.2	End-to-end runtime sequence for one production query.	13
3.3	Container deployment topology from <code>docker-compose.yml</code>	14
3.4	Continuous MLOps lifecycle used to operate and improve the platform. . . .	17
4.1	Screenshot Slot 1: Frontend Landing Page	26
4.2	Screenshot Slot 2: Chat Interaction	26
4.3	Screenshot Slot 3: API Documentation	27
4.4	Screenshot Slot 4: Docker Services	27
4.5	Screenshot Slot 5: LangSmith and Evaluation	27

List of Tables

3.1	Major software dependencies in the project.	10
3.2	Current data assets in the <code>data/</code> directory.	14
3.3	Document metadata schema used for traceability and debugging.	15
3.4	Core retrieval/generation controls from runtime configuration.	15
4.1	Implementation breakdown by MLOps stage.	21
4.2	Data representation metrics sheet (fill after run).	23
4.3	Operational performance representation template.	23
4.4	RAG evaluation report structure.	24
A.1	Operational measurement sheet for final viva/demo.	32

List of Abbreviations

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
CSV	Comma Separated Values
EDA	Exploratory Data Analysis
JSON	JavaScript Object Notation
LLM	Large Language Model
MLOps	Machine Learning Operations
NLP	Natural Language Processing
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
SaaS	Software as a Service
SQL	Structured Query Language
TXT	Plain Text File Format
UI	User Interface

Chapter 1

Introduction

1.1 Overview

The recent adoption of generative AI in SaaS products has changed expectations for customer support, internal search, onboarding, and knowledge management. End users now expect natural language interfaces that can answer complex questions from proprietary business documents. Standard LLM chat interfaces, however, are often not sufficient for enterprise usage because they can hallucinate and lack direct access to current domain data.

Retrieval-Augmented Generation (RAG) addresses this by attaching a retrieval stage before generation. Instead of asking the model to answer from its internal pretraining alone, the system retrieves relevant chunks from a curated document store and injects them into the prompt context. This approach improves factual grounding and reduces unsupported claims.

The project implemented in this report is an AI SaaS chatbot backend based on this RAG principle. The system is engineered with practical deployment goals: support for multiple LLM providers, local vector persistence, reproducible ingestion, memory across multi-turn conversations, error handling, and quantitative evaluation. The implementation is designed to operate as a realistic backend service rather than as an isolated notebook experiment.

1.2 Machine Learning Context

Machine learning is the computational framework that allows systems to discover patterns from data and make predictions or decisions with minimal explicit rule coding. In chatbot systems, machine learning appears in multiple layers:

- Embedding models convert text into dense vectors capturing semantic meaning.

- Retrieval models rank stored chunks by vector similarity.
- Generative LLMs synthesize responses conditioned on retrieved evidence.
- Evaluation models score response quality using objective metrics.

Traditional software can match keywords, but ML-driven semantic encoding allows conceptual matching, where user intent and retrieved content can align even when exact words differ. In this project, a Hugging Face embedding model is used to create vector representations and ChromaDB stores and indexes those vectors for fast similarity lookup.

1.3 Machine Learning Strategies Relevant to This Work

1.3.1 Supervised Learning

Supervised learning uses labeled data to learn mappings from input to output. In the context of this project, supervised logic appears indirectly in pretrained embeddings and LLMs that were trained on massive supervised and instruction-tuned corpora. Although this project does not train an LLM from scratch, it deploys supervised-learning products as inference components.

1.3.2 Unsupervised and Self-Supervised Learning

Unsupervised and self-supervised paradigms underpin representation learning for text embeddings. The system benefits from this through pretrained embeddings that encode semantic relationships without requiring task-specific labels from the project team.

1.3.3 Reinforcement and Preference Optimization (Background)

Modern LLM behavior is often shaped by reinforcement learning from human feedback or preference optimization methods. This report includes this topic as background because the deployed providers (Gemini and Groq-hosted open models) include alignment behavior from such training. This matters for practical chatbot performance, safety, and tone.

1.4 Problem Statement

Organizations often maintain support content in fragmented formats: PDF manuals, CSV exports, spreadsheets, and internal notes. Users ask questions in natural language, but

static search interfaces return weak results, while direct LLM chats may return ungrounded answers.

The project aims to solve this gap by building a retrieval-grounded chatbot backend that can:

- ingest multi-format documents,
- index and retrieve context semantically,
- generate concise and context-bounded answers,
- preserve conversational continuity with memory,
- remain observable and testable in production-like workflows.

1.5 Objectives

The principal objectives are:

1. Build a robust ingestion pipeline for common enterprise document formats.
2. Create a persistent local vector database for semantic retrieval.
3. Implement a multi-provider LLM layer with fallback behavior.
4. Expose the full pipeline through a FastAPI interface.
5. Add reproducibility through Docker, testing, and operational scripts.
6. Add quality and observability frameworks using Ragas and LangSmith.

1.6 Scope and Deliverables

The project scope includes backend design and implementation of the RAG system and core MLOps capabilities. Frontend work exists as a separate React artifact but is not the primary focus of this report. The final deliverables are source code modules, configuration files, technical documentation, automated tests, and this formal project report.

1.7 Report Organization

The remaining chapters are organized as follows:

- Chapter 2 reviews related literature and industrial patterns in RAG systems.
- Chapter 3 details methodology, architecture, algorithms, and module design.
- Chapter 4 explains implementation, testing, results, and discussion.
- Chapter 5 presents conclusions and future enhancements.
- Appendices include representative source code and supporting artifacts.

Chapter 2

Literature Survey

2.1 Related Work in Retrieval-Augmented Generation

RAG emerged as a practical bridge between parametric memory (inside LLM weights) and non-parametric memory (external documents). Earlier question-answering systems used sparse lexical retrieval and template-based response logic. With transformer-based embeddings, dense retrieval became dominant for semantic search use cases. Recent SaaS products combine dense retrieval with LLM generation to provide support copilots, internal assistants, and domain-specific chat interfaces.

The key insight from literature is that generation quality depends strongly on retrieval quality. Even a strong LLM performs poorly when irrelevant or incomplete context is retrieved. Therefore, robust systems focus on data chunking, metadata tracking, retriever configuration, and evaluation metrics.

2.2 Document Ingestion and Chunking Strategies

A recurring theme in research and engineering practice is chunking strategy. Very small chunks may lose semantic continuity; very large chunks can exceed token budgets and reduce retriever precision. Recursive splitters that prioritize paragraph and sentence boundaries typically outperform naive fixed-character segmentation in readability and grounding fidelity.

This project adopts `RecursiveCharacterTextSplitter` with configurable chunk size and overlap. This allows balancing retrieval granularity against context cohesion.

2.3 Vector Databases for Semantic Retrieval

Vector databases such as ChromaDB, Qdrant, Weaviate, and Pinecone are now standard components in RAG stacks. They support indexing and nearest-neighbor lookup over high-dimensional embeddings. Local embedded vector stores are useful in early-stage development due to simplicity, while managed or distributed stores are preferred in high-scale enterprise deployment.

The present implementation uses ChromaDB in persistent local mode. This is appropriate for educational and prototype-to-SME deployments and supports reproducibility in offline development.

2.4 LLM Provider Strategy and Reliability

Production systems increasingly avoid hard dependence on one provider. Service outages, quota limits, and model lifecycle changes can disrupt user-facing chatbot operations. A common strategy is primary-plus-fallback model routing. Error categorization (quota, unavailable model, transient upstream error) is critical for deciding whether to retry, fail fast, or switch models.

The system described in this report includes explicit model fallback sequencing and provider-agnostic exception handling. This design follows resilient service principles from modern AI operations.

2.5 Observability and Evaluation Practices

A major weakness in many student and startup RAG systems is lack of observability. Without trace-level visibility, debugging poor responses is difficult. Platforms such as LangSmith provide retrieval traces, prompts, and output chains for root-cause analysis.

Similarly, objective evaluation frameworks are essential. Ragas provides measurable metrics such as faithfulness, answer relevancy, context precision, and context recall. Literature shows that such metrics enable controlled iteration and regression detection.

2.6 Testing and MLOps Lifecycle

Classic ML pipelines were often evaluated offline and deployed with limited integration testing. Current MLOps standards emphasize continuous validation: data ingestion checks, unit

tests, behavior regressions, and deployment reproducibility. Containerization, infrastructure automation, and CI hooks convert research prototypes into operational systems.

This project aligns with that direction by combining backend modules, tests, documentation, Docker deployment files, and operational automation scripts.

2.7 Research Gaps and Relevance to This Project

Key gaps identified across related work include:

- limited grounding controls in naive chatbot implementations,
- insufficient operational reliability under provider failures,
- missing quality baselines and poor experiment traceability,
- lack of ingestion automation for frequently updated corpora.

The implemented system addresses these gaps with explicit prompt constraints, model fallback, evaluation and tracing support, and ingestion orchestration. While not yet a fully hardened global-scale architecture, it represents a comprehensive, production-minded educational implementation.

Chapter 3

Methodology and System Design

3.1 Existing System

The baseline problem pattern observed in many early chatbot projects includes:

- one-time manual ingestion,
- ad hoc scripts without persistent memory,
- no fallback for LLM failures,
- limited or absent quality metrics,
- weak deployment reproducibility.

Such systems may work during demonstrations but fail under realistic usage where document sets evolve, model APIs face limits, and multi-turn context matters.

3.2 Proposed System

The proposed architecture is a layered RAG backend with the following modules:

1. **Ingestion Layer:** Loads PDF, TXT, CSV, XLSX, and JSON sources.
2. **Chunking Layer:** Applies recursive splitting with overlap.
3. **Embedding and Indexing Layer:** Generates vectors and persists in ChromaDB.
4. **Retrieval Layer:** Fetches top-k semantically relevant chunks.
5. **Generation Layer:** Uses provider-selected LLM with fallback sequence.

6. **Memory Layer:** Stores chat history in SQLite by chat_id.
7. **API Layer:** Exposes endpoints for health and question-answer flow.
8. **Operations Layer:** Adds Docker, observability, evaluation, and automation.

3.3 Functional Requirements

- Ingest heterogeneous document formats.
- Build and persist vector embeddings.
- Retrieve relevant context for user queries.
- Generate concise responses grounded in retrieved data.
- Support multi-turn conversation memory.
- Provide API endpoints for external clients.
- Expose diagnostic behavior for failures.

3.4 Non-Functional Requirements

- Maintainability through modular Python files.
- Extensibility for additional providers and retrievers.
- Reliability via retry/fallback behavior.
- Reproducibility via Dockerized environment.
- Observability via traces and evaluation metrics.

3.5 Software and Hardware Requirements

3.5.1 Software Requirements

- Python 3.11
- FastAPI, Uvicorn

- LangChain ecosystem modules
- ChromaDB
- Hugging Face embedding dependencies
- Pytest for testing
- Docker and Docker Compose

3.5.2 Hardware Requirements

- Minimum 8 GB RAM (16 GB recommended for smooth local model operations)
- Multi-core CPU
- SSD storage for vector index and local data
- Stable internet for cloud LLM providers

3.5.3 Core Libraries Used

Library	Purpose
langchain / langchain-core	Prompt chaining and retrieval pipeline orchestration
langchain-groq	Groq LLM integration
langchain-google-genai	Gemini model and embedding integrations
langchain-chroma	Vector store connection
pydantic-settings	Configuration management from environment variables
fastapi	HTTP API framework
sqlite3	Persistent local chat memory
ragas	RAG quality evaluation metrics
watchdog	File watching for automated ingestion
pytest	Automated test execution

Table 3.1: Major software dependencies in the project.

3.6 Programming Language Rationale

Python was selected because it offers mature libraries for NLP, rapid API development, and MLOps tooling. Its ecosystem includes direct support for RAG components and easy integration with vector stores, LLM providers, and testing frameworks.

3.7 System Architecture

3.7.1 Logical Architecture View

The platform is intentionally separated into operational layers so each engineering concern remains independently maintainable. The presentation layer handles user interaction and request initiation. The API layer enforces schema validation and HTTP contracts. The orchestration layer handles retrieval logic, prompt assembly, and fallback decisions. The knowledge layer is responsible for vector retrieval and conversational state persistence. DataOps automates ingestion and corpus refresh, while the MLOps layer provides reproducibility, observability, and quality governance.

This layered split is not cosmetic. It creates explicit isolation boundaries between data movement, model inference, and platform reliability. That allows independent hardening of each subsystem and reduces blast radius during failures. For example, ingestion failures should not crash serving traffic, and model quota failures should trigger controlled fallback rather than unstructured API errors.

3.7.2 Request-Response Runtime Sequence

At runtime, each user query is processed through a deterministic sequence. Memory is loaded first to preserve multi-turn context. Retrieval then uses both the current question and recent conversational turns to disambiguate follow-up prompts. The generation payload is constructed only after retrieval, ensuring context-bounded outputs. The error path is explicitly classified, so only retryable conditions trigger fallback models. Finally, response metadata and messages are persisted for continuity and auditing.

This runtime sequence forms the primary control path of the application and is the core unit of production reliability. Any regression in this sequence directly affects correctness, latency, and user trust. For that reason, the flow is instrumented and designed to be observable and testable.

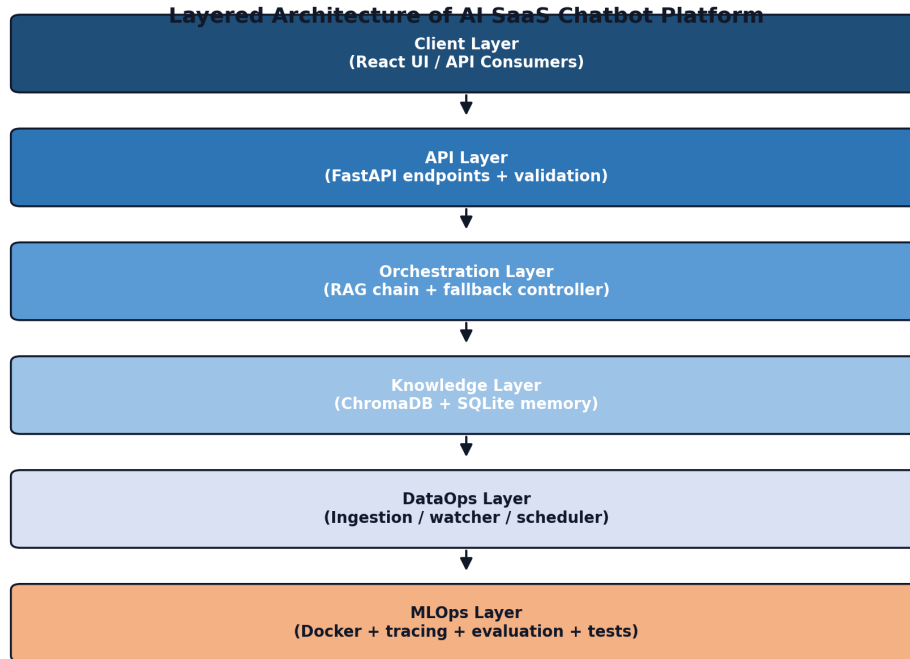


Figure 3.1: Layered architecture used by the AI SaaS chatbot platform.

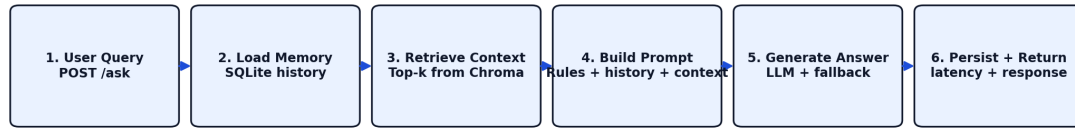
3.7.3 Deployment Architecture with Docker

The deployment is container-centric and isolates API and vector database responsibilities:

- **api container:** Runs FastAPI/Uvicorn, RAG orchestration, retrieval logic, and memory interactions.
- **chroma container:** Runs the vector database service with persistence enabled.
- **bridge network:** Internal service-to-service communication using service names.
- **health checks:** Container-level checks ensure service readiness before traffic.
- **volumes:** Persistent mounts for `chroma_db`, `data`, and `chat_memory.db`.

The deployment topology is designed around two core services with explicit contracts: an API compute service and a vector database service. Network-level isolation keeps service communication private to the compose network, while mounted volumes preserve durable state across container restarts. This model makes local development and production-like validation behaviorally consistent.

Runtime Request Sequence (Per Query)



Fallback path is activated only for quota/model/unavailable/transient provider errors.

Figure 3.2: End-to-end runtime sequence for one production query.

3.7.4 Repository and Filesystem Architecture

```
AI_SAAS(ChatBot) copy/
+-- backend/                # Core platform modules
|   +-- api.py              # FastAPI interface
|   +-- rag_engine.py       # Retrieval + generation orchestration
|   +-- ingest.py           # Multi-format ingestion and chunking
|   +-- vector_store.py     # ChromaDB integration
|   +-- memory.py           # SQLite persistent chat history
|   +-- evaluation.py       # Ragas quality evaluation
|   +-- observability.py    # LangSmith tracing integration
|   +-- file_watcher.py     # Event-based ingestion automation
|   +-- ingest_orchestrator.py # watch/once/schedule/daemon modes
+-- frontend/              # React frontend client
+-- data/                  # Ingestion source corpus
+-- docs/                  # Design, testing, MLOps documentation
+-- tests/                 # Pytest validation suite
+-- Dockerfile             # Multi-stage Python image build
+-- docker-compose.yml     # API + Chroma service orchestration
+-- report/                # Project report source and PDF
```

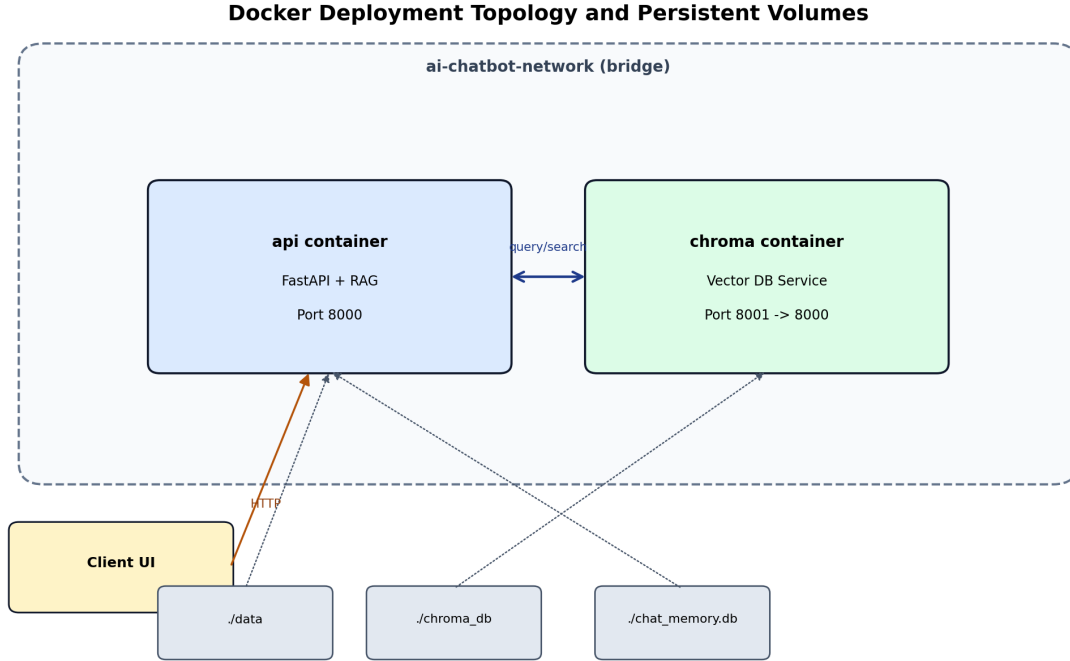



Figure 3.3: Container deployment topology from `docker-compose.yml`.

3.7.5 Data Representation Model

(A) Source corpus representation in this repository:

File	Type	Size (bytes)	Role in Pipeline
product_faq.txt	TXT	5339	Domain knowledge source for semantic chunks
golden_dataset.json	JSON	3210	Evaluation/benchmark question set
test.csv	CSV	53	Tabular ingestion validation sample

Table 3.2: Current data assets in the `data/` directory.

(B) Internal metadata representation per document chunk:

(C) Retrieval and generation control representation:

3.7.6 Industrial Complexity in the Implemented System

The complexity of this project is concentrated in integration and operations rather than a single algorithm:

Field	Type	Meaning
page_content	string	Raw text payload used for embedding and prompt context
source	string	Absolute or relative source file path
row	integer	Row index when data comes from CSV/XLSX records
item	integer	Item index when data comes from JSON arrays

Table 3.3: Document metadata schema used for traceability and debugging.

Control Parameter	Current Value	Operational Impact
chunk_size	500	Controls retrieval granularity and context density
chunk_overlap	50	Preserves continuity across adjacent chunks
top_k	4	Number of retrieved chunks per query
temperature	0.1	Low-variance, factual response style
memory_max_messages	6	Limits history depth for prompt efficiency

Table 3.4: Core retrieval/generation controls from runtime configuration.

1. **Provider abstraction with fallback:** supports multiple model backends and retry routing based on error classification.
2. **Context-grounded response enforcement:** system prompt constraints plus retrieval-driven answer policy reduce unsupported outputs.
3. **Stateful conversational memory:** persistent SQL-backed history with bounded replay for follow-up resolution.
4. **DataOps automation:** watcher and scheduler modes enable continuous corpus refresh without manual CLI intervention.
5. **Reproducible deployment:** multi-container orchestration with health checks and shared persistent volumes.

6. **Observability and measurable quality:** LangSmith traces and Ragas metrics provide diagnosable, testable model behavior.
7. **Failure-domain handling:** explicit API error mapping for quota, model availability, and upstream provider failures.

The practical difficulty of this system lies in coordinating multiple failure-prone subsystems under one user-facing SLA: document pipelines, vector retrieval, model providers, state persistence, and HTTP serving. In production-like behavior, these components do not fail symmetrically. A model endpoint can rate-limit while ingestion is healthy; retrieval can succeed while answer generation fails; API uptime can remain high while answer quality drifts. A serious implementation must account for these mixed-mode states and preserve graceful degradation.

For this reason, the platform is treated as an iterative operations loop rather than a one-time model deployment. Each release cycle includes ingestion validation, retrieval verification, latency checks, trace inspection, and evaluation score review before promotion. This lifecycle orientation is the reason the project incorporates automation, monitoring, and quality measurement directly into the repository rather than as post-development add-ons.

3.7.7 MLOps Lifecycle Diagram

The lifecycle in Figure above captures the operating principle used in this project: data ingestion feeds indexing; indexing feeds retrieval and serving; serving generates traces and quality metrics; metrics drive deployment and configuration iteration; iteration loops back into data and retrieval improvements. This closed-loop model is essential for RAG systems because data and user behavior evolve continuously even when application code remains unchanged.

3.8 Algorithmic Flow

3.8.1 Data Ingestion Algorithm

1. Scan target directory for supported extensions.
2. Load files through format-specific loaders.
3. Convert rows/items to documents with metadata.
4. Split into overlapping chunks recursively.

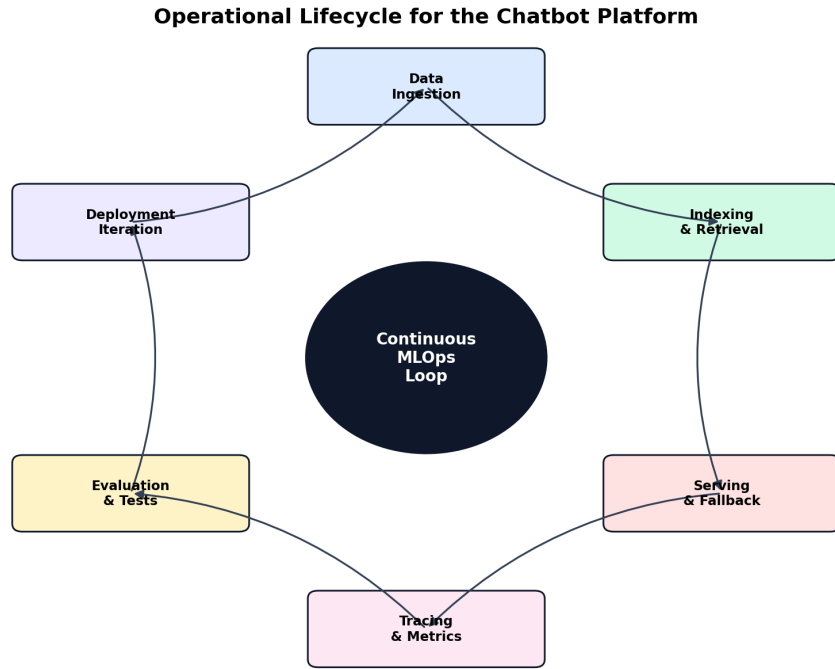


Figure 3.4: Continuous MLOps lifecycle used to operate and improve the platform.

5. Push chunks to vector store with embeddings.

3.8.2 Retrieval and Generation Algorithm

1. Accept user question and optional chat history.
2. Build retrieval query using recent conversation turns.
3. Retrieve top-k chunks from ChromaDB.
4. Create prompt with system rules, history, and context.
5. Invoke primary model.
6. If error is retryable (quota/unavailable/transient), attempt fallback model.
7. Return generated response or mapped API error.

3.8.3 Memory Management Algorithm

1. Receive chat_id with user request.
2. Read recent messages from SQLite table.
3. Pass messages into prompt as history placeholder.
4. After answer generation, append user and assistant messages.
5. Enforce configured history limits for retrieval and generation payload size.

3.9 Module-Wise Design

3.9.1 Configuration Module

`backend/config.py` centralizes environment-driven settings such as provider, model names, fallback models, chunk size, top-k retrieval, temperature, and memory path.

3.9.2 Ingestion Module

`backend/ingest.py` handles multi-format data loading and recursive chunking. It supports PDF, TXT, CSV, Excel, and JSON input formats.

3.9.3 Vector Store Module

`backend/vector_store.py` provides functions to create/load ChromaDB instances and ingest chunks with consistent embedding configuration.

3.9.4 RAG Engine Module

`backend/rag_engine.py` composes retrieval, prompting, model invocation, fallback logic, and domain-level exception mapping.

3.9.5 API Module

`backend/api.py` exposes `/health` and `/ask` chatbot endpoints. It includes CORS policy, startup pre-warm, validation, and structured error responses.

3.9.6 Memory Module

`backend/memory.py` manages SQLite-based chat memory with indexed lookup by `chat_id` and ordered replay for prompt history.

3.9.7 Evaluation Module

`backend/evaluation.py` integrates Ragas metrics to evaluate faithfulness, relevancy, context precision, and context recall.

3.9.8 Observability Module

`backend/observability.py` enables LangSmith tracing when API credentials are configured.

3.9.9 Automation Modules

`backend/file_watcher.py` and `backend/ingest_orchestrator.py` provide continuous/scheduled ingestion patterns.

3.10 Design Decisions and Tradeoffs

- **Local embeddings over cloud embeddings by default:** lower cost and offline-friendly, with slightly lower quality in some domains.
- **Chroma local persistence:** simple and reproducible, but not ideal for distributed multi-instance scale.
- **Prompt-grounded output rule:** reduces hallucination risk, may occasionally return insufficient-answer responses.
- **Fallback across model candidates:** improves availability, but can create response style variance.

Chapter 4

Implementation, Results, and Discussion

4.1 Implementation Workflow and MLOps Staging

Implementation was treated as a platform engineering process, not a one-pass coding exercise. Each module was developed with explicit integration contracts so behavior could be validated incrementally under realistic runtime conditions. This approach reduced late-stage coupling risk and made root-cause identification practical when failures appeared across data, retrieval, or provider boundaries.

The staging model below reflects how work was sequenced in practice: data pipeline stabilization first, then retrieval/generation orchestration, then reliability controls, and finally observability and evaluation loops. This order matters because later stages depend on confidence in upstream data and retrieval behavior.

Implementation followed an engineering lifecycle that maps to practical MLOps phases:

4.2 Container Build and Runtime Engineering

4.2.1 Docker Image Design

Containerization in this project is primarily about deterministic behavior across machines and runs. The same code, dependencies, and startup script must produce the same serving profile regardless of host environment. Multi-stage build design reduces runtime image footprint while preserving operational dependencies needed by the inference stack.

The API image is built using a multi-stage Dockerfile:

Phase	Primary Components	Engineering Outcome
DataOps	<code>ingest.py</code> , <code>file_watcher.py</code> , <code>ingest_orchestrator.py</code>	Automated multi-format ingestion and refresh pipeline
ModelOps	<code>embedder.py</code> , <code>vector_store.py</code> , <code>rag_engine.py</code>	Retrieval-grounded generation with model fallback control
ServingOps	<code>api.py</code> , <code>memory.py</code>	Stateful API surface for low-latency chat operations
ReliabilityOps	fallback logic, mapped exceptions, startup pre-warm	Controlled failure domains and lower cold-start latency
ObservabilityOps	<code>observability.py</code> , tracing configuration	Trace-level visibility of retrieval and generation path
EvaluationOps	<code>evaluation.py</code> , golden dataset support	Quantitative RAG quality baseline and regression framework
PlatformOps	Dockerfile + docker-compose + tests	Reproducible runtime and verification workflow

Table 4.1: Implementation breakdown by MLOps stage.

- **Builder stage:** installs compile dependencies and Python packages.
- **Runtime stage:** copies built artifacts into a smaller execution image.
- **Health contract:** a healthcheck probes the API endpoint to support orchestration safety.
- **Deterministic environment:** pinned dependencies reduce host-machine drift.

4.2.2 Service Orchestration Design

Service orchestration is designed to protect the serving path from initialization races and hidden state issues. The API process is started only after vector infrastructure is healthy, and persistent volumes ensure that index data and chat memory survive restarts. This improves confidence in repeated experiments and deployment rehearsals.

The `docker-compose.yml` introduces service isolation and dependency ordering:

- API service depends on healthy Chroma service before startup.

- Dedicated bridge network isolates east-west traffic between services.
- Persistent volume strategy separates code, mutable data, and index storage.
- Restart policy supports crash recovery and long-running service behavior.

4.2.3 Operational Commands Used in Deployment

```
docker-compose build
docker-compose up -d
docker-compose ps
docker-compose logs -f api
docker-compose logs -f chroma
docker-compose down
```

4.3 Data Representation and Evidence Tables

4.3.1 Corpus and Chunking Representation

For retrieval systems, representation quality is as critical as model selection. The system therefore tracks data at multiple layers: raw files, parsed records, semantic chunks, and indexed vectors. These measurements provide early warning signals when ingestion changes degrade downstream answer quality.

The pipeline stores raw files and transforms them into metadata-linked semantic units. The following template is used for reporting run-time representation metrics after execution:

4.3.2 Retrieval and Generation Performance Sheet

4.4 Evaluation and Observability Lens

4.4.1 RAG Quality Evaluation

Quality evaluation is framed as an operational gate rather than a documentation artifact. Any change in chunking, retrieval configuration, or model routing is expected to be reflected in metric movement. The report template below supports repeatable comparisons across versions and encourages evidence-based release decisions.

Metric	Value	How Measured	Interpretation
Total input files	_____	find data -type f	Corpus breadth
Total documents loaded	_____	ingestion logs	Parsed record count
Total chunks created	_____	chunking logs	Retrieval granularity
Avg chunk length	_____	post-split stats	Prompt packing efficiency
Vector count in store	_____	Chroma inspection	Index completeness

Table 4.2: Data representation metrics sheet (fill after run).

Scenario	Latency (ms)	Provider Used	Observation
Cold start request	_____	_____	Effect of startup pre-warm
Warm request (normal)	_____	_____	Steady-state performance
Fallback triggered	_____	_____	Resilience under failure
Long-context query	_____	_____	Prompt assembly overhead

Table 4.3: Operational performance representation template.

The evaluation module applies Ragas metrics across questions, retrieved contexts, model answers, and ground truths. This creates a measurable contract for model quality. The following report format is used:

4.4.2 Traceability and Incident Debugging

Metrics indicate that quality changed; traces explain why it changed. Both are required for robust operation of RAG systems where failures may be cross-layer. A retrieval mismatch can look like generation hallucination, and provider instability can look like random latency spikes. Trace-level visibility separates these cases and shortens debugging cycles.

LangSmith tracing enables root-cause analysis across the full path:

- inspect retriever inputs and selected chunks,
- inspect final prompt payload and history injection,

Metric	Score	Interpretation
Faithfulness	_____	Whether generated claims are grounded in retrieved evidence
Answer Relevancy	_____	Whether answer aligns with user intent
Context Precision	_____	Ratio of useful chunks among retrieved chunks
Context Recall	_____	Coverage of required facts in retrieval output
Aggregate Score	_____	Overall quality baseline for this release

Table 4.4: RAG evaluation report structure.

- inspect LLM response timings and provider/model route,
- compare successful and failed runs for regression diagnosis.

4.5 Validation Strategy

4.5.1 Automated Testing

The automated suite focuses on deterministic invariants that should never regress: file loading correctness, schema constraints, chain behavior contracts, and fallback decision logic. Provider-dependent stochastic outputs are validated through controlled manual runs and observability checks, preventing flaky automated gates.

The test suite is structured to validate ingestion and RAG engine behavior under deterministic conditions. Core coverage focus:

- file format loading behavior (valid and malformed inputs),
- response contract behavior of the ask pipeline,
- fallback-path and failure-handling logic through mocks,
- regression-safe behavior for refactors.

4.5.2 Manual Validation Checklist

1. Run ingestion and verify vector store persistence.

2. Start API and verify health endpoint.
3. Execute `/ask` with single-turn and multi-turn prompts.
4. Validate fallback behavior by forcing provider-side failure conditions.
5. Validate Dockerized run and compare with local run behavior.

4.6 Complex Engineering Outcomes

The most significant outcome is reliable integration across heterogeneous subsystems. The platform combines ingestion automation, semantic indexing, context-grounded generation, provider fallback, persistent chat memory, and observability loops into one service boundary. This integrated behavior is what makes the system operationally meaningful.

The following implemented capabilities represent the highest complexity zones:

- **Fallback-aware generation path:** error categorization and model routing within one request cycle.
- **State-aware retrieval:** chat history influences retrieval query formulation for follow-up coherence.
- **Persistent memory integration:** SQL-based conversation continuity, not only in-memory state.
- **Background ingestion modes:** watch/schedule/daemon support for continuously changing corpora.
- **Operationally ready container stack:** service boundaries, health checks, persistent volumes, and restart behavior.
- **Quality governance layer:** measurable quality metrics and trace instrumentation built into the repository.

4.7 Evidence Placeholders for Final Submission

The following screenshot slots are intentionally left for final run-time evidence insertion:

Screenshot Slot 1: Frontend Landing Page

React UI loaded and connected to backend

(Insert screenshot after running the system)

Figure 4.1: Screenshot Slot 1: Frontend Landing Page

Screenshot Slot 2: Chat Interaction

User query, assistant answer, and multi-turn follow-up

(Insert screenshot after running the system)

Figure 4.2: Screenshot Slot 2: Chat Interaction

4.8 Current Limitations and Hardening Plan

- Chroma local persistence should move to distributed service mode for multi-instance deployments.
- AuthN/AuthZ and rate limiting are pending for production perimeter security.
- Hybrid retrieval (vector + lexical) and reranking can improve difficult query classes.
- CI/CD integration should enforce automated tests and evaluation checks on every merge.

Screenshot Slot 3: API Documentation

FastAPI Swagger page at `/docs`

(Insert screenshot after running the system)

Figure 4.3: Screenshot Slot 3: API Documentation

Screenshot Slot 4: Docker Services

`docker-compose ps` and container health status

(Insert screenshot after running the system)

Figure 4.4: Screenshot Slot 4: Docker Services

Screenshot Slot 5: LangSmith and Evaluation

One trace view plus Ragas metrics summary output

(Insert screenshot after running the system)

Figure 4.5: Screenshot Slot 5: LangSmith and Evaluation

Chapter 5

Conclusion and Future Work

5.1 Conclusion

This project successfully delivers an AI SaaS chatbot backend built on Retrieval-Augmented Generation principles. The implementation integrates ingestion, chunking, embeddings, vector retrieval, grounded generation, persistent memory, API serving, testing, tracing, and evaluation in one coherent workflow.

The core contribution is not a single model innovation but a practical engineering integration of multiple AI and software subsystems. This aligns with current industry demand where reliability, observability, and maintainability matter as much as raw model capability.

From an academic perspective, the work demonstrates a mature understanding of applied machine learning system design. It translates concepts from NLP and ML operations into executable architecture and documented engineering decisions.

5.2 Future Work

The following enhancements are proposed for enterprise-grade hardening:

1. Introduce JWT/OAuth-based authentication and role-based access control.
2. Upgrade vector storage to a distributed service (Qdrant or Weaviate).
3. Add hybrid retrieval (BM25 + dense vectors) and reranking.
4. Move ingestion jobs to asynchronous queues (Celery/Redis or Temporal).
5. Integrate CI pipelines for tests and periodic Ragas regression checks.

6. Add cost telemetry and token budgeting across providers.
7. Expand frontend experience with robust session and admin tooling.

5.3 Final Statement

The project has achieved a strong MLOps-capable baseline and is ready for the next phase of security and scale improvements. It provides both educational and practical value, and it is suitable for submission as a comprehensive final-year engineering report.

Bibliography

- [1] LangChain Documentation. <https://python.langchain.com/docs/>
- [2] Chroma Documentation. <https://docs.trychroma.com/>
- [3] FastAPI Documentation. <https://fastapi.tiangolo.com/>
- [4] Ragas Documentation. <https://docs.ragas.io/>
- [5] LangSmith Documentation. <https://docs.smith.langchain.com/>
- [6] Pydantic Settings Documentation. https://docs.pydantic.dev/latest/concepts/pydantic_settings/
- [7] Docker Documentation. <https://docs.docker.com/>
- [8] SQLite Documentation. <https://www.sqlite.org/docs.html>
- [9] Mark Treveil et al., *Introducing MLOps*. O'Reilly Media, 2020.
- [10] Patrick Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” NeurIPS, 2020.

Appendix A

Deployment Runbook and Operational Evidence

A.1 Environment and Startup Checklist

1. Verify environment variables: provider key(s), model selection, and persistence paths.
2. Confirm corpus availability in `data/`.
3. Run ingestion and verify vector persistence.
4. Start API (local or Docker mode).
5. Verify service health and API docs.
6. Run query and confirm memory persistence.

A.2 Deployment Command Log Template

```
# Local mode
python backend/ingest.py
bash scripts/start.sh
curl http://127.0.0.1:8000/
curl http://127.0.0.1:8000/docs

# Docker mode
docker-compose build
docker-compose up -d
```

```
docker-compose ps
docker-compose logs -f api
```

A.3 Runtime Observation Sheet

Observation	Measured Value	Notes
API startup time	_____	Includes pre-warm initialization
Average response latency	_____	5-query moving average
Fallback activation count	_____	Error-handling validation
Ingestion throughput	_____	Files or chunks per run

Table A.1: Operational measurement sheet for final viva/demo.

Appendix B

Screenshots and Submission Attachments

Final screenshots are primarily placed in Chapter 4 under “Evidence Placeholders for Final Submission.” Use this appendix only for overflow images if additional pages are required by the review panel.

B.1 Plagiarism and Similarity Report Placeholder

Attach the official institutional plagiarism report in this section before final submission.

B.2 Journal/Conference Paper Placeholder

Attach the extended paper version in this section if required by department policy.